

## ¿Qué es el Comparator?

El Comparator es una interfaz funcional en Java que permite comparar dos objetos para definir su orden. Se utiliza para definir órdenes personalizados o alternativos en comparación con el orden natural de una clase.

### ¿Por qué usar Comparator?

1. **Orden Alternativo:** Permite definir múltiples maneras de ordenar los objetos de una clase.
  - Ejemplo: Ordenar por nombre, edad, o cualquier atributo deseado.
2. **No Modifica la Clase Original:** La lógica de comparación está separada, por lo que no necesitas alterar la clase que estás comparando.
3. **Mayor Flexibilidad:** Es ideal cuando necesitas ordenar una misma clase en diferentes contextos.

### Casos de Uso de Comparator

1. **Gestión de Inventarios:** Ordenar productos por precio, cantidad o popularidad.
2. **Sistemas Académicos:** Clasificar estudiantes por calificación, nombre o matrícula.
3. **Aplicaciones de Reservas:** Comparar vuelos por precio, duración o cantidad de escalas.
4. **Procesamiento de Datos:** Clasificar registros en bases de datos por fechas o campos específicos.

### Cómo Usar Comparator en Java: Operación y Retorno

El método `compare(T o1, T o2)` de la interfaz `Comparator` devuelve un **entero** que indica la relación entre dos objetos `o1` y `o2`:

- **Valor Negativo (< 0):** `o1` debe preceder a `o2`.
- **Valor Cero (0):** `o1` y `o2` son equivalentes (igualdad lógica).
- **Valor Positivo (> 0):** `o1` debe seguir a `o2`.

### ¿Cómo Opera Internamente?

1. **Clasificación Ascendente:**
  - Si `compare(o1, o2)` devuelve un valor negativo, `o1` se coloca antes que `o2`.
2. **Clasificación Descendente:**
  - Si `compare(o1, o2)` devuelve un valor positivo, `o1` se coloca después que `o2`.
3. **Igualdad Lógica:**
  - Si devuelve 0, ambos objetos son tratados como equivalentes.

## Ejemplo Simple: Orden Ascendente por Edad

```
class Persona {
    String nombre;
    int edad;

    public Persona(String nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
    }

    @Override
    public String toString() {
        return "Persona{nombre='" + nombre + "', edad=" + edad + "}";
    }
}

import java.util.Comparator;

public class Main {
    public static void main(String[] args) {
        Persona p1 = new Persona("Alice", 30);
        Persona p2 = new Persona("Bob", 25);

        // Comparator para comparar por edad ascendente
        Comparator<Persona> comparadorPorEdad = (o1, o2) -> Integer.compare(o1.edad, o2.edad);

        // Comparar dos objetos
        int resultado = comparadorPorEdad.compare(p1, p2);
        if (resultado < 0) {
            System.out.println(p1.nombre + " es menor que " + p2.nombre);
        } else if (resultado > 0) {
            System.out.println(p1.nombre + " es mayor que " + p2.nombre);
        } else {
            System.out.println(p1.nombre + " tiene la misma edad que " + p2.nombre);
        }
    }
}
```

El Comparator usa `Integer.compare(o1.edad, o2.edad)` para comparar las edades.

Dado que  $25 < 30$ , el método devuelve un número negativo, lo que indica que Bob debe preceder a Alice.

## Ordenar una Lista con Comparator

Cuando se usa Comparator con métodos como Collections.sort(), el método compare se llama automáticamente para comparar pares de objetos y determinar su posición en la lista.

### Ejemplo: Ordenar Personas por Edad

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        List<Persona> personas = new ArrayList<>();
        personas.add(new Persona("Alice", 30));
        personas.add(new Persona("Bob", 25));
        personas.add(new Persona("Charlie", 35));

        // Comparator para ordenar por edad ascendente
        Comparator<Persona> comparadorPorEdad = (o1, o2) -> Integer.compare(o1.edad, o2.edad);

        // Ordenar la lista usando el Comparator
        Collections.sort(personas, comparadorPorEdad);

        // Mostrar lista ordenada
        System.out.println("Personas ordenadas por edad: " + personas);
    }
}
```

## Ordenar una Lista con Comparator

Cuando se usa Comparator con métodos como Collections.sort(), el método compare se llama automáticamente para comparar pares de objetos y determinar su posición en la lista.

### Ejemplo: Ordenar Personas por Edad

```
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;

public class Main {
    public static void main(String[] args) {
        List<Persona> personas = new ArrayList<>();
        personas.add(new Persona("Alice", 30));
        personas.add(new Persona("Bob", 25));
        personas.add(new Persona("Charlie", 35));

        // Comparator para ordenar por edad ascendente
        Comparator<Persona> comparadorPorEdad = (o1, o2) -> Integer.compare(o1.edad, o2.edad);

        // Ordenar la lista usando el Comparator
        Collections.sort(personas, comparadorPorEdad);

        // Mostrar lista ordenada
        System.out.println("Personas ordenadas por edad: " + personas);
    }
}
```

## Ordenar por Múltiples Criterios

El Comparator también permite encadenar criterios de comparación usando el método `thenComparing`.

### Ejemplo: Ordenar Primero por Edad y Luego por Nombre

```
public class Main {
    public static void main(String[] args) {
        List<Persona> personas = new ArrayList<>();
        personas.add(new Persona("Alice", 30));
        personas.add(new Persona("Bob", 30));
        personas.add(new Persona("Charlie", 25));

        // Comparator por edad y luego por nombre
        Comparator<Persona> comparador = Comparator.comparingInt((Persona p) -> p.edad)
            .thenComparing(p -> p.nombre);

        // Ordenar la lista usando el Comparator
        personas.sort(comparador);

        // Mostrar lista ordenada
        System.out.println("Personas ordenadas por edad y nombre: " + personas);
    }
}
```

## Comparadores Personalizados con Uso Real

### 1. Ordenar por Fecha Más Reciente

```
import java.time.LocalDate;
import java.util.Comparator;

class Registro {
    String descripcion;
    LocalDate fecha;

    public Registro(String descripcion, LocalDate fecha) {
        this.descripcion = descripcion;
        this.fecha = fecha;
    }

    @Override
    public String toString() {
        return "Registro{descripcion='" + descripcion + "', fecha=" + fecha + "}";
    }
}

public class Main {
    public static void main(String[] args) {
        List<Registro> registros = new ArrayList<>();
        registros.add(new Registro("Evento A", LocalDate.of(2023, 5, 20)));
        registros.add(new Registro("Evento B", LocalDate.of(2022, 11, 15)));
        registros.add(new Registro("Evento C", LocalDate.of(2024, 3, 10)));

        // Comparador para ordenar por fecha más reciente
        Comparator<Registro> comparadorPorFecha = (r1, r2) -> r2.fecha.compareTo(r1.fecha);

        // Ordenar registros
        registros.sort(comparadorPorFecha);

        // Mostrar registros ordenados
        System.out.println("Registros ordenados por fecha más reciente: " + registros);
    }
}
```

## ¿Por qué Usar Comparator en Lugar de equals(), ==, o contains() en Java?

Java proporciona múltiples maneras de comparar objetos, como ==, equals(), y contains(). Sin embargo, en muchos casos, el uso de Comparator ofrece ventajas significativas, especialmente cuando se trata de ordenar o manejar estructuras complejas. A continuación, analizamos las diferencias y por qué Comparator suele ser una mejor opción en varios escenarios.

### Comparación de Métodos de Comparación

| Método     | Propósito                                                                                  | Ventajas                                                    | Limitaciones                                                  |
|------------|--------------------------------------------------------------------------------------------|-------------------------------------------------------------|---------------------------------------------------------------|
| ==         | Comprueba si dos referencias apuntan al mismo objeto en memoria.                           | Rápido, directo.                                            | No compara contenido, solo referencias.                       |
| equals()   | Compara el contenido de dos objetos (según la implementación del método en la clase).      | Útil para verificar equivalencia lógica.                    | Requiere sobrescribir equals() en la clase; un solo criterio. |
| contains() | Comprueba si una colección contiene un elemento igual según la implementación de equals(). | Sencillo para verificar pertenencia en colecciones simples. | Limitado al criterio definido por equals().                   |
| Comparator | Define reglas personalizadas de comparación entre objetos.                                 | Flexible, permite múltiples criterios, desacopla la lógica. | Más trabajo inicial para configurar comparadores.             |

## ¿Por Qué Usar Comparator en Lugar de Otros Métodos?

### 1. Mayor Flexibilidad

- Mientras que equals() y contains() funcionan bien para comparar objetos según un único criterio predefinido, Comparator permite definir múltiples criterios sin modificar la clase base.
- Ejemplo: Comparar productos por precio, luego por nombre, y finalmente por fecha de adición.

### 2. Desacoplamiento de la Lógica

- La lógica de comparación con Comparator no está incrustada en la clase del objeto, sino en una clase o lambda independiente. Esto evita modificar la clase original para cada nuevo criterio de comparación.

### 3. Ordenación Personalizada

- Comparator es fundamental para ordenar colecciones según criterios específicos, algo que equals() o == no pueden lograr.
- Ejemplo: Ordenar una lista de estudiantes por calificaciones o nombres alfabéticamente.

### 4. Compatibilidad con APIs de Java

- Muchas estructuras y métodos de Java, como Collections.sort() y TreeSet, dependen de Comparator para definir el orden de los elementos.

### 5. Reutilización

- Un Comparator puede ser reutilizado en diferentes contextos y aplicaciones sin modificar la clase que compara.

## Casos Prácticos y Comparación

### Caso 1: Comparar Referencias (==) vs. Contenido (Comparator)

- ==:
  - Verifica si dos referencias apuntan al mismo objeto.
  - No es útil para comparar el contenido lógico de los objetos.
- Comparator:
  - Compara el contenido de los objetos según un criterio definido.

### Ejemplo: Comparar dos objetos Persona por edad usando Comparator

```

Persona p1 = new Persona("Alice", 30);
Persona p2 = new Persona("Bob", 30);

// Comparar referencias (incorrecto para comparar contenido)
System.out.println(p1 == p2); // false (son diferentes referencias)

// Comparar contenido usando Comparator
Comparator<Persona> porEdad = Comparator.comparingInt(p -> p.edad);
System.out.println(porEdad.compare(p1, p2)); // 0 (tienen la misma edad)

    System.out.println("Retrocediendo: " + listIterator.previous());
}

```

### Caso 2: Comparar con equals() vs. Comparator

- **equals():**
  - Requiere sobrescribir el método equals() en la clase.
  - Sólo admite un criterio de comparación.
- **Comparator:**
  - Permite definir múltiples criterios de comparación de manera externa.

### Ejemplo: Comparar por nombre usando Comparator

```

Persona p1 = new Persona("Alice", 30);
Persona p2 = new Persona("Alice", 25);

// equals() compara solo según la implementación predeterminada (nombre y edad deben coincidir)
System.out.println(p1.equals(p2)); // false

// Comparator permite comparar sólo por nombre
Comparator<Persona> porNombre = Comparator.comparing(p -> p.nombre);
System.out.println(porNombre.compare(p1, p2)); // 0 (los nombres son iguales)

```



### Caso 3: Búsqueda en Colecciones (contains()) vs. Uso de Comparator

- **contains():**
  - Funciona con equals() y verifica si un objeto existe en una colección.
  - No permite criterios personalizados de comparación.
- **Comparator:**
  - Facilita la búsqueda en colecciones ordenadas según criterios específicos.

#### Ejemplo: Búsqueda por criterios personalizados en un TreeSet

```

TreeSet<Persona> personas = new TreeSet<>(Comparator.comparing(p -> p.nombre));
personas.add(new Persona("Alice", 30));
personas.add(new Persona("Bob", 25));

// contains() usa el Comparator para buscar por nombre
System.out.println(personas.contains(new Persona("Alice", 40))); // true (nombre coincide, edad no importa)
    
```

#### Ventajas Clave de Comparator Sobre Otros Métodos

| Característica                 | == | equals() | contains() | Comparator |
|--------------------------------|----|----------|------------|------------|
| Compara Referencias            | ✓  | X        | X          | X          |
| Compara Contenido              | X  | ✓        | ✓          | ✓          |
| Admite Múltiples Criterios     | X  | X        | X          | ✓          |
| No Requiere Modificar la Clase | ✓  | X        | X          | ✓          |
| Ordenación Personalizada       | X  | X        | X          | ✓          |

#### Casos de Uso Exclusivos de Comparator

1. **Ordenación Compleja en Sistemas Reales**
  - Ordenar vuelos por precio, escalas, duración y compañía.
  - Clasificar productos por popularidad, calificación y precio.
2. **Jerarquías de Comparación**
  - Comparar estudiantes primero por calificaciones, luego por asistencia y finalmente por nombre.
3. **Optimización en Colecciones**
  - Implementar búsquedas rápidas en estructuras ordenadas como TreeSet o TreeMap.